Name: Student ID:

Agentic AI for Business and FinTech (SEEM5660)
Individual Homework 04, Due date: 1 December, 23:59 HKT.

Individual homework. You are encouraged to discuss with others; however, you must submit your own work.

1 Overview

An equity-research desk receives dozens of short questions from its analysts every day: *What was the net accounts receivable of company X at the end of 2024? Which of X and Y held more cash? What target price does broker Z set for X?* The answers are somewhere in the desk's library of annual reports, earnings-call transcripts and broker research reports. Tutorial 7 showed how a **Retrieval-Augmented Generation (RAG)** assistant answers such questions: it retrieves a few passages and lets an LLM answer from them.

When a RAG assistant gives a wrong answer, the cause is usually not the LLM but the **retrieval**: the right passage never reached it, or a passage about the wrong company or the wrong year did. In this homework the LLM that writes the answers (the *generator*) is fixed and given to you. You build everything that decides *what the generator sees*: parsing the HTML and its tables, chunking with metadata, understanding the question, retrieving at most five chunks, and deciding when the library cannot answer the question at all.

Part	What you do	Points	Graded
Task 1.1	Convert HTML tables to text	8	automatically
Task 1.2	Chunk a document, with metadata	7	automatically
Task 2.1	Return valid retrieval results	5	automatically
Task 2.2	Parse the question (company, year, document type)	10	automatically
Task 2.3	Retrieval recall	15	automatically
Task 3	Abstention: recognise unanswerable questions	10	automatically
Task 4	End-to-end answer accuracy	20	automatically
Task 5	Ablation study and report	15	by the TAs
Task 6	Reflection	10	by the TAs
	Total	100	

Tasks 1 to 4 build on each other, but each is graded on its own: e.g. a good `table_to_text` earns its 8 points even if your retrieval is weak.

If you encounter any problems with this homework, please inform the TA immediately: Hongzheng Yang (hzyang05@gmail.com).

2 Setup

Fork `https://github.com/hz0567/FTEC5660-HW4`, then (Python 3.10 or newer):

```
git clone <https://github.com/hz0567/FTEC5660-HW4>
cd FTEC5660-HW4
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
echo "DEEPSEEK_API_KEY=your_key_here" > .env
python3 hw4.py --data-folder public_test
```

The repo contains `hw4.py` (the five functions you write, followed by the provided code: the models, the generator, the loaders, the runner and the scoring) and `public_test/` (40 documents of 10 fictional companies, 60 questions and their answers; Appendix A). Use your own DeepSeek API key (`https://platform.deepseek.com`), as in Tutorial 7. No other key is needed. Never commit `.env`; it is already in `.gitignore`. The starter runs before you add any code. It scores about 5 points (it answers `NOT_FOUND` to every question).

Models. The LLM allowed is `deepseek-v4-flash`, through `make_llm()`, which counts your tokens. Do not create your own model or API clients. You may use the LLM in `parse_query`, `build_index` and `retrieve` (e.g. for query rewriting, HyDE, reranking or an answerability check), but not in `table_to_text` or `chunk_document`: the runner checks this, and those tasks score 0 if a model is called.

Retrieval without embeddings. Unlike Tutorial 7, this homework uses **no embedding model**, neither through an API nor locally (no `sentence-transformers`, no vector store). Retrieval is lexical: BM25 (`rank_bm25`, or LangChain's `BM25Retriever`), TF-IDF, keyword rules, and filtering on your metadata. On short factual questions about named companies and years, this is enough for high recall, and every run is cheap, fast and reproducible. Any other pure-Python or standard scientific package (e.g. NumPy, scikit-learn's `TfidfVectorizer`) is allowed; add it to `requirements.txt`. No other data source (web search, hosted APIs, ...) is allowed.

3 The Data and the Pipeline

3.1 Documents and questions

`corpus/` holds one HTML file per document, e.g. `doc_038.html`. The file names say nothing about the content. Each company has four documents (Appendix A.2):

Document	`doc_type`, `fiscal_year`	Content
Annual report FY2024	`annual_report`, 2024	Narrative, governance, financial statements and notes; every table has the columns 2024 and 2023.
Annual report FY2025	`annual_report`, 2025	The same, with the columns 2025 and 2024.
Q4 FY2025 earnings call	`earnings_call`, 2025	Transcript: rounded results, the outlook for FY2026, a share repurchase programme, Q&A.
FY2025 results review	`research_report`, 2025	A broker's note: rating, target price, estimates for FY2026 and FY2027, peer table.

The public corpus has about 73,000 tokens: small enough to read, so look at a few documents before you start. (A real desk's library would not fit into a prompt. Here the corpus is small to keep your experiments cheap, and the generator only ever sees five chunks.) The questions come in four categories:

Category	Example	Public
`fact`	Who was Halcyon Freight's Chief Financial Officer at the end of FY2025? What 12-month target price does BuyFin Capital set for Kestrel Semiconductor's shares in its FY2025 results review, in USD?	12
`table`	What were Northgate Robotics' net accounts receivable at 31 December 2025, in USD millions?	22
`comparison`	Which company held more cash and cash equivalents at 31 December 2024: Northgate Robotics or Northgate Payments?	12
`unanswerable`	What was Halcyon Air Cargo's revenue in FY2024, in USD millions?	14
	Total	60

Things to watch for.

- **Similar names.** Some companies share a word: Northgate Robotics and Northgate Payments, Brightwater Software and Brightwater Energy.
- **Two years per table.** The FY2025 annual report also shows the FY2024 figures, and the FY2023 figures appear only in the FY2024 report. A figure that appears in two documents has the same value in both.
- **Companies not in the corpus.** Questions about Halcyon Air Cargo or Northgate Robotic Systems look like questions about Halcyon Freight or Northgate Robotics, but those companies are not in the corpus.
- **People change.** Some companies changed their CFO or their auditor in 2025. The FY2025 report names both the old and the new one.
- **Rounded and forecast figures.** The earnings call and the letter to shareholders round figures ("revenue of about USD 2.4 billion"). The exact figure is in the tables. A research report gives its new FY2026 estimate next to its previous one, and its peer table lists other companies.
- **Unanswerable questions.** About one question in four asks about a company that is not in the corpus, a year that is not covered (e.g. FY2021), or a figure that is not disclosed (e.g. lobbying expenses).

3.2 The pipeline

```
for every document:   chunk_document(file_name, html)  -> chunks with metadata  (uses
    table_to_text)
once:                 build_index(all chunks)            -> index
for every question:   retrieve(index, question)          -> at most 5 of those chunks, or []
                      generate(question, chunks)         -> the answer      (fixed, provided)
```

The generator (`generate()` in `hw4.py`, prompt in Appendix B) is `deepseek-v4-flash` at temperature 0. It sees the question and the retrieved chunks, each with a header built from the chunk's metadata, e.g.

```
[1] source: doc_038.html | company: Northgate Robotics | fiscal_year: 2025 | doc_type:
    annual_report
Accounts receivable, net | 31 December 2025: 492.1 | 31 December 2024: 477.3
...
```

It answers with one number, one name, or `NOT_FOUND`. If `retrieve` returns `[]`, the generator is not called and the answer is `NOT_FOUND`.

3.3 Rules

- Write only the five functions marked `### YOUR CODE HERE` and your own helpers above them. Before grading, we replace everything below the line *Provided code below* with the original, so changes there have no effect.
- `retrieve` may only return chunks that `chunk_document` produced (compared by `page_content`). The runner drops any other chunk and keeps at most 5.
- The text of a chunk must come from its own document: the document's text, the output of `table_to_text` for its tables, and optionally a short header with the metadata and the section title. No text from other documents, no text written by an LLM, no hard-coded content.

- Do not hard-code file names, company names, questions or answers. We grade on a private corpus of other companies.
- Each run must finish within 30 minutes, including building the index.

4 Task 1: Parsing and Chunking (15 points)

4.1 Task 1.1: Tables to text (8 points)

Most numbers sit in HTML tables. Naive text extraction (e.g. `get_text()`) turns a table into a stream of numbers in which the link between a value, its row and its column is lost (Tutorial 7, Section 3.1). Write `table_to_text(table_html)`, which receives one `<table>` element as a string and returns text in which **every value is on one line with its row label and its column header**, one table row per line, and the caption (which holds the unit) is included. For the income statement of Northgate Robotics' FY2025 report, for example:

```
Consolidated income statement for the year ended 31 December (USD millions, except per-share
    data)
Revenue | 2025: 2,710.4 | 2024: 2,629.1
Cost of revenue | 2025: (1,762.4) | 2024: (1,721.3)
...
```

Grading. `tables.json` holds 12 tables, one of each kind (income statement, balance sheet, cash flow, segments, regions, receivables, borrowings, employees, board, executives, a research report's financial summary and its peer table), each with the list of its cells (row label, column header, value). A cell counts if one line of your output (at most 400 characters) contains its row label, its column header and its value, unchanged, and no other row label of the table. The score of a table is the share of its cells that count, halved if the caption is missing. Task 1.1 is worth 8× the mean score over the 12 tables of the private set. Case and whitespace are ignored. Your format is free otherwise ("`Revenue (2025): 2,710.4`" is fine too); a Markdown table is not, because its rows do not contain the column headers.

4.2 Task 1.2: Chunking with metadata (7 points)

Write `chunk_document(file_name, html)`, which returns the chunks of one document as a list of LangChain `Document`s. Every chunk must have at most 1,500 characters (`MAX_CHUNK_CHARS`) and the metadata:

Key	Value
`source`	The file name, e.g. `"doc_038.html"`.
`company`	The company the document is about, e.g. `"Northgate Robotics"` (a legal suffix such as "Inc." is optional). For a research report, the company under review, not the broker.
`fiscal_year`	An `int`: the fiscal year of the annual report, or 2025 for the earnings call and the research report (they review FY2025).
`doc_type`	`"annual_report"`, `"earnings_call"` or `"research_report"`.

All four are on the first lines of each document (Appendix A.2). You may add more metadata (e.g. the section title). Use `table_to_text` for the tables. How you split is up to you: fixed size with overlap, by section, one chunk per table, parent and child chunks, ...

Grading. A document passes if all its chunks have at most 1,500 characters and the right metadata, and its chunks together still contain at least 90% of a sample of the document's figures and names (as printed, e.g. `2,710.4` or `Rachel Nyberg`; `docs.json` lists them for the public documents as `probes`). Task 1.2 is worth 7× the share of the 40 private documents that pass. The runner prints why a document fails.

5 Task 2: Retrieval (30 points)

Write `build_index(chunks)`, called once with the chunks of the whole corpus, and `retrieve(index, question)`, which returns at most 5 of those chunks, best first. Return `[]` when you decide the question cannot be answered (Task 3).

Example (`public_test`, question `q021`). *"Which company held more cash and cash equivalents at 31 December 2024: Northgate Robotics or Northgate Payments?"* Two facts are needed: Northgate Robotics' cash at the end of 2024 (215.7, in its FY2024 report `doc_028.html` or in the prior-year column of its FY2025 report `doc_038.html`) and Northgate Payments' (114.5, in `doc_039.html` or `doc_011.html`). A naive retriever returns five similar-looking balance sheet chunks, often all of one company, or of both companies but for 2025. A good retriever finds out that the question names two companies and one year, and retrieves for each company separately.

5.1 Task 2.1: Valid retrieval (5 points)

5× the share of answerable private questions for which `retrieve` returns a non-empty list of at most 5 chunks, all of them produced by `chunk_document`, without raising an error.

5.2 Task 2.2: Query parsing (10 points)

Most retrieval errors come from not knowing what the question is about. Write `parse_query(question)`, which returns a dict with three fields (use it in `retrieve`, e.g. for metadata filtering):

Field	Rule
`companies`	A list of every company the question asks about, as named in the question (order, case and a legal suffix do not matter). A company that is not in the corpus is still listed. Brokers (e.g. BuyFin Capital) and audit firms are not listed.
`fiscal_year`	The fiscal year the question names, as an `int`: "FY2024" and "31 December 2024" both give 2024. A question names at most one year; `None` if it names none (e.g. "In which city is Halcyon Freight headquartered?").
`doc_type`	`"earnings_call"` if the question asks about an earnings call; `"research_report"` if it asks about a broker's rating, target price, forecast or results review; otherwise `"annual_report"`.

For example, *"What FY2026 revenue does Meridian Equity Research forecast for Sablewood Retail, in USD millions?"* gives {`"companies": ["Sablewood Retail"], "fiscal_year": 2026, "doc_type": "research_report"`}. Note that `fiscal_year` is the year the question names, not the year of the document that holds the answer: the FY2023 revenue is in the FY2024 report.

Grading. The score of a question is the share of the three fields that are right. Task 2.2 is worth 10× the mean over the private questions. Rules, regular expressions, the LLM, or a combination are all fine. You may use what `chunk_document` found (e.g. the list of companies in the corpus), since the runner calls it first.

5.3 Task 2.3: Retrieval recall (15 points)

A question is a **hit** if, for every fact it needs, one of the retrieved chunks comes from a document that states the fact and contains the fact as printed there (e.g. `215.7` as a whole number, or the CFO's name). `ground_truth.json` lists these (document, text) pairs as the `evidence` of each public question. **Recall@5** is the share of answerable questions that are hits. Task 2.3 is worth

$$15 \times \min\Big(1,\ \max\Big(0,\ \frac{\text{recall@5} - 0.5}{0.9 - 0.5}\Big)\Big),$$

i.e. 0 points up to 50% recall, full points from 90%, linear in between (e.g. 80% gives 11.25 points).

Tips.

- Look at what your retriever returns before you change anything: `results.csv` lists the retrieved files and the hit of every question.
- BM25 is bad at telling Northgate Robotics from Northgate Payments (both contain "Northgate") and 2024 from 2025 (every report mentions both years). Metadata is good at it.
- A comparison needs a chunk for each company. Five chunks of one company are a miss.

Keywords that may help: chunk size and overlap, contextual chunk headers, parent-document retrieval, metadata filtering and self-query retrieval, entity resolution, BM25 and its parameters (k_1, b), tokenisation of numbers and names, stop words and stemming, field boosting, query expansion with synonyms ("cash" and "cash equivalents", "headcount" and "employees"), LLM-as-reranker, query rewriting and decomposition, HyDE (a hypothetical answer as the BM25 query).

6 Task 3: Abstention (10 points)

A retriever always finds *something*: for "What was Halcyon Air Cargo's revenue in FY2024?" it happily returns Halcyon Freight's income statement, and the generator may then answer with Halcyon Freight's revenue. Your `retrieve` should return `[]` when the corpus cannot answer the question, and chunks otherwise. With

- *kept* = the share of answerable questions for which `retrieve` returns chunks, and
- *rejected* = the share of unanswerable questions for which it returns `[]`,

Task 3 is worth $10 \times \max(0,\ \text{kept} + \text{rejected} - 1)$. Always returning chunks (kept 100%, rejected 0%) or never (0%, 100%) gives 0 points. Kept 98% and rejected 70% give 6.8 points. Which questions are unanswerable is described in Section 3.1. For figures that are not disclosed, you need to check whether the retrieved chunks actually contain what the question asks for (e.g. with a score threshold or an LLM check).

7 Task 4: End-to-End Accuracy (20 points)

The fixed generator answers every private question from what you retrieved. Task 4 is worth 20× the answer accuracy. An answer is correct if:

- **number questions**: it contains exactly one number, within max(1% of the true value, 0.05) of the true value;
- **name questions**: it contains the expected name (case, punctuation and titles such as "Dr." are ignored) and does not also name the other option of a comparison;
- **unanswerable questions**: it is `NOT_FOUND`. `NOT_FOUND` to an answerable question is wrong.

The generator is the same for everybody, so this measures how well your retrieval serves it. Good recall is necessary but not sufficient: the generator can still pick the wrong column or the wrong company if your chunks are ambiguous, which is why the metadata header and the format of your table text matter.

Evaluation of Tasks 1 to 4

We run your code once on a private set of 40 documents of 10 other fictional companies, 60 questions, 12 tables and the metadata of its documents. It has the same document structure and the same question categories in the same proportions. Only the companies, the numbers and some wording differ. If a run fails because of an API error on our side, we run it again.

Your submission must run end-to-end and produce `results.csv`.
A run that crashes or exceeds 30 minutes scores zero on Tasks 1 to 4. Test it from a fresh clone before you submit. We start every run from a fresh clone, so your code must build its index from the corpus it is given. Do not commit an index built on the public corpus.

8 Task 5: Ablation Study and Report (15 points)

Which of your design choices actually helped? Starting from a naive baseline, change **one component at a time** and measure each configuration on `public_test`:

- **B0, naive RAG**, as in Tutorial 7, Section 2: the text of the document from BeautifulSoup's `get_text()` (no `table_to_text`), split with `RecursiveCharacterTextSplitter()`, the required metadata, BM25 top-5 over the whole corpus (`rank_bm25` with lower-cased whitespace tokens), no query parsing, never `[]`.
- **V1, ...**: each adding or changing one component relative to the row above it (e.g. `table_to_text`, metadata filtering with `parse_query`, a better tokeniser, a contextual chunk header, LLM reranking, per-company retrieval for comparisons, an abstention rule).

Write your report in a "Homework 4 solution" section of your `README.md`: a figure of your pipeline and one paragraph describing it (your parsing, chunking, metadata, index, retrieval and abstention, and why), then the ablation table and its analysis.

Grading. A complete table that matches your code and your `results.csv` (6 points); variants that change one thing each, with an explanation of why each change helped or did not (5 points); the error analysis (2 points); the pipeline figure and description (2 points).

9 Task 6: Reflection (10 points)

Submit your reflection following these questions: Imagine that ten years from now, everyone has access to powerful personal AI agents capable of writing, programming, conducting research, managing businesses, and making sophisticated decisions.

- What kind of person would you want to be in that world?
- Which of your current ambitions would still matter?
- What would you want to be known for beyond your productivity and professional achievements?
- What are you willing to struggle for, even when an AI agent could make the struggle unnecessary?
- What skills, qualities, or experiences would still make you valuable?
- If your value is no longer determined primarily by what you can do, how will you define who you are?
- What should you start doing differently tomorrow?

No restrictions on the format. You can write/draw/or submit an AI-generated submission.

10 Submission

Submit your fork, a public GitHub repo named `FTEC5660-HW4`, by entering your GitHub link on Blackboard. We grade the last commit before the deadline. The repo contains:

- `hw4.py` with your five functions,
- `requirements.txt`, with any extra packages you use added,
- any script you use for Task 5,
- `README.md` with your report (Task 5) and your reflection (Task 6, or a link to it).

Do not put any API key in your code: we run your code with our own DeepSeek key. Do not commit index files (e.g. `*.pkl`). Keep the repo public during grading. For students unfamiliar with GitHub, we recommend this tutorial: `https://www.youtube.com/watch?v=-RZ03WHqkaY`.

A The Data

A.1 Files

```
public_test/
  corpus/doc_001.html ... doc_040.html   # 10 companies x 4 documents
  questions.json                         # {"q001": "How much cash ...?", ...}
  ground_truth.json                      # the answers and the evidence (Tasks 2.2 to 4)
  tables.json                            # 12 tables with their cells (Task 1.1)
  docs.json                              # the metadata and probes of every document (Task 1.2)
```

`ground_truth.json` gives, for every question, its category, its answer type (`number`, `name` or `not_found`), the answer, the `evidence` (one list per fact the question needs, of every [document, text as printed] that states it) and the expected `query` for Task 2.2. Name answers may list `aliases` (also accepted) and `reject` (the other option of a comparison):

```
"q051": {"category": "table", "type": "number", "answer": 492.1,
         "evidence": [[["doc_038.html", "492.1"]]],
         "query": {"companies": ["Northgate Robotics"], "fiscal_year": 2025, "doc_type": "
    annual_report"}},
"q021": {"category": "comparison", "type": "name", "answer": "Northgate Robotics",
         "aliases": ["Northgate Robotics Inc."], "reject": ["Northgate Payments"],
         "evidence": [[["doc_028.html", "215.7"], ["doc_038.html", "215.7"]],
                      [["doc_011.html", "114.5"], ["doc_039.html", "114.5"]]],
         "query": {"companies": ["Northgate Robotics", "Northgate Payments"], "fiscal_year":
    2024, ...}},
"q005": {"category": "unanswerable", "type": "not_found", "answer": "NOT_FOUND", "evidence":
    [],
         "query": {"companies": ["Halcyon Freight"], "fiscal_year": 2022, "doc_type": "
    annual_report"}}
```

`tables.json` is a list of {`"source", "caption", "html", "cells"`}, where `cells` is a list of [row label, column header, value]. `docs.json` maps every file name to {`"company", "fiscal_year", "doc_type", "probes"`}. The private set has the same files; you never see its `ground_truth.json`, but `questions.json`, `tables.json` and `docs.json` are passed to the same runner.

A.2 The documents

Every document is one HTML file with a navigation bar, headings (`<h1>` to `<h3>`), paragraphs, lists and real `<table>` elements. All amounts are in USD millions unless the text says otherwise.

Annual report. The `<h1>` is the company's legal name (e.g. "Northgate Robotics Inc."), followed by "Annual Report 2025", the fiscal year end, the exchange and ticker, the headquarters and the industry. Then: 1. Letter to shareholders (rounded figures); 2. Business overview (segments, regions, employees); 3. Risk factors; 4. Management's discussion and analysis; 5. Corporate governance (board, executive officers, CFO changes, the independent auditor); 6. Financial statements (income statement, balance sheet, cash flow statement); 7. Notes (revenue by segment and by region, receivables, borrowings, employees by function).

Earnings call. The `<h1>` is "*legal name* (*exchange*: *ticker*)", followed by "Fourth Quarter and Full Year 2025 Earnings Conference Call" and the date (February 2026). Then the participants, the presentation (CEO and CFO: rounded results, the share repurchase programme, the revenue guidance for FY2026) and the Q&A with analysts.

Research report. A first line "*broker* — Equity Research — *industry*", then the `<h1>` "*legal name* (*ticker*)" and "FY2025 results review: ...". Then the rating (BUY, HOLD or REDUCE), the 12-month target price, the share price, an investment summary, the estimate changes (the new and the previous

FY2026 revenue estimate), a financial summary table (FY2024A, FY2025A, FY2026E, FY2027E), the valuation with a peer table, and the risks.

B Provided Code in `hw4.py`

- `make_llm()`: a `ChatDeepSeek` for `deepseek-v4-flash` that counts its tokens in `USAGE`. Thinking mode is off by default (as in Tutorial 7); with `thinking=True`, the temperature is ignored and thinking tokens count as output.
- `USAGE` and `GENERATOR_USAGE`: your and the generator's token counters; `.summary()` returns the tokens so far and the estimated cost.
- `generate(question, chunks)` and `format_context(chunks)`: the fixed generator and the context it sees. Call them yourself to inspect what the generator does with your chunks.
- `load_corpus`, `load_json`, and the scoring functions used by the runner: `table_score`, `document_problems`, `parse_score`, `retrieval_hit`, `answer_correct`.

```
llm = make_llm()                              # deepseek-v4-flash, thinking off
print(llm.invoke("Hello").content)
print(USAGE.summary())                        # your tokens so far, estimated cost
```

The generator's prompt:

```
You answer questions about company reports, using only the context below.

Context:
{context}

Question: {question}

Rules:
- Use only the context. The header of each passage says which document it comes from.
- Check that the passage you use is about the company and the fiscal year that the question
    asks about.
- If the context does not contain the answer, reply exactly NOT_FOUND.
- For a number, reply with that one number only, in the unit the question asks for, without
    units, words or other numbers (for example: 1913.4).
- For a name (a person, a company, a city, an audit firm or a rating), reply with the name only
    .

Answer:
```

The header of each chunk shows its `source`, `company`, `fiscal_year` and `doc_type` (each cut to 100 characters), taken from the chunk that `chunk_document` produced.